Indoor Coverage Path Planning

Bo Shi, Yao Chu and Ling Xin

***Abstract*— Coverage Path Planning (CPP) describes the process of generating a robot trajectory that fully covers a specific area or volume. Applications include mobile cleaning robots, lawn mowing robots, and agricultural harvesters. Despite various studies on this topic, research combining CPP with autonomous modeling is scarce. This paper adopts a method that integrates path planning with autonomous modeling, enabling the robot to explore the space online and perform complete path coverage.**

I. INTRODUCTION

Our research presents an innovative method that combines CPP with autonomous modeling. This integration allows robots to autonomously explore and model an environment online and subsequently execute a path that achieves complete coverage. Utilizing the Robot Operating System (ROS), our approach harnesses the capabilities of the Turtlebot3 Burger, a robot well-suited for indoor use due to its compact size and versatility.

The core components of our methodology include:

Mapping: Utilizing Gmapping-SLAM (Simultaneous Localization and Mapping) to develop accurate maps of the environment.

Localization: Also use the ability of localization of gmapping to determine the robot's position within the mapped area.

Path Planning: Implementing a coverage path planning algorithm that ensures efficient and thorough coverage of the entire area.

The implementation of this system is carried out in a simulated environment using Gazebo, providing a controlled setting for testing and validation. By enabling the robot to autonomously map and navigate through its environment, our approach aims to significantly enhance the operational efficiency and effectiveness of service robots in various applications.

II. METHODS

A. Robot Platform and Simulation Environment

To conduct our experiments, we utilized the Turtlebot3 Burger robot. This robot was selected due to its compact size, making it ideal for the limited indoor environments typically encountered in cleaning tasks. The Turtlebot3 Burger is equipped with essential sensors, including a 360-degree LiDAR, an inertial measurement unit (IMU), and wheel encoders. These sensors are critical for performing tasks such as mapping, localization, and navigation.

The experiments were conducted in a simulated environment using Gazebo. Gazebo is a powerful robot simulation tool that allows for realistic and controlled testing conditions. The simulated environment consisted of a 6m x 4m (Figure 1) space designed to mimic a typical indoor environment. This space included various static obstacles such as walls, furniture, and narrow passages to replicate real-world conditions.

For three-dimensional visualization we use RViz which is primarily used with the Robot Operating System (ROS). It allows users to visualize sensor data, robot states, and other related information. We use it to display robot models, laser scans, scanned map, global map, published goal points, planned path and already covered area of the robot. This helps us intuitively understand the robot's surroundings and its behaviour.

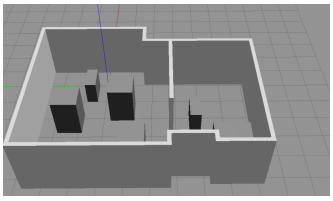

Fig. 1. Gazebo-world

B. Mapping and localization

The mapping process began with the initialization phase. The Turtlebot3 Burger was placed at a starting location within the simulated environment. We initialized the Robot Operating System (ROS) and the turtlebot3 simulation, launching the necessary nodes (move-base, explore-lite and slam-gmapping) for mapping and control. The robot then started autonomous exploration of the environment using its LiDAR sensor to gather range data.

For mapping, we employed the Gmapping-SLAM (Simultaneous Localization and Mapping) algorithm and explore lite framework. Those algorithms process the sensor data collected by the LiDAR to build a 2D occupancy grid map incrementally while simultaneously keeping track of its current position within that map as well as navigate the robot through the environment. The robot will continuously updated the map by detecting and mapping features and exploring the unknown area. The exploration continued until the robot had traversed the entire environment, resulting in a comprehensive and accurate map. See figure 2.

*This project was assisted by Fraunhofer IPA.

[1]This work was guided by Prof. Richard Bormann Richard Bormann@ipa.fraunhofer.de

[2]The authors are studying Autonomous Systems at the University of Stuttgart, 70569 Stuttgart, Germany <first name>.<last name>@stud.uni-stuttgart.de

Fig. 2. Mapping

C. Exploring

For exploring the unknown environment we use the explore-lite framework. Explore Lite is a lightweight exploration framework based on the Robot Operating System (ROS), primarily used for autonomous navigation and environmental exploration. Explore Lite is designed to provide an efficient and straightforward solution suitable for robotic systems with limited computational resources. This framework can autonomously plan paths and adjust in real-time according to changes in the environment. Robots can then automatically explore and map unknown environments. It can seamlessly integrate with existing ROS packages and tools, such as SLAM and the navigation stack.

After it detects the boundaries of unexplored areas, it will determine new exploration directions. In the mean time it calculates safe navigation paths based on the current map and sensor data. With continuously published new goal points the robot will be guided to unexplored areas. It can also adjusts the exploration strategy in real-time based on sensor feedback.

D. Coverage Path Planning (CPP)

With the environment mapped and the robot localized, we proceeded to the coverage path planning phase. The environment was divided into a grid, with each cell representing a small region that the robot needed to cover. We used A* algorithm for path planning.

The A* algorithm is a popular pathfinding and graph traversal algorithm that is often used in many fields of computer science due to its performance and accuracy. The A* algorithm is employed here to modify a generated coverage path so that it navigates around obstacles. We first generate an initial path that covers the area without considering obstacles. This path is generated based on the occupancy grid and might include points that are inside obstacles or very close to them. Then we take the initially generated path and use the A* algorithm to modify each consecutive point of this path to avoid obstacles.

The coverage path planning algorithm generated a path that ensured the robot would visit each cell while avoiding area with obstacles. The path planning steps included defining a starting point, setting the density of the grid and paths, as well as using the A* algorithm to calculate the best path that maximizes the cover area while bypassing all the obstacles. See figure 3.

The planned path was then uploaded to the goal-publisher node. This node will send each goal point in proper order according to the received path to robot's navigation stack. The robot began following the path, making real-time adjustments to avoid obstacles and ensure complete coverage of the area. This dynamic path adjustment was facilitated by the move-base navigation stack, which used real-time sensor data to detect and navigate around new obstacles.

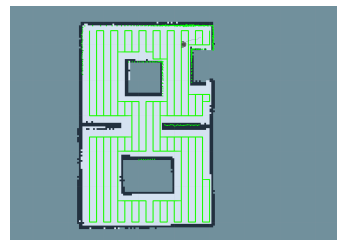

Fig. 3. Generated Path

E. System Integration and Execution

Our system was composed of multiple ROS nodes, each responsible for different tasks such as mapping, localization, exploring and coverage path planning. Key ROS launch files include coverage planning.launch for initiating the covering operation, gazebo.launch for starting the gazebo simulation and turtlebot3-slam.launch for initializing the turtlebot3 simulation. These launch files coordinated the various components of the system, ensuring seamless operation.

To achieve obstacle avoidance, we use the move-base navigation stack as the module that controls the robot to track a full-coverage path. The system architecture is shown in figure 4. Real-time sensor data allowed the robot to detect and avoid obstacles that were not present during the initial mapping and also assist the real-time navigation. A goal point publication node dynamically updated the robot's goals as it progressed through the coverage task, enabling continuous and adaptive coverage.

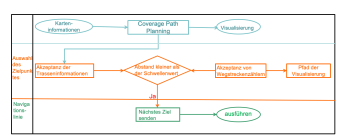

Fig. 4. Robot Control Flowchart

III. Experiment and Evaluation

After we constructed our workspace (three major folders, each containing launch files, 3D world map and two python files: one for path generating and one for goal publishing) in ubuntu system, the experiment is quiet simple. We started experiment by launching the launch file and the gazebo turtlebot as well as rviz started automatically. The robot shown in rviz began to build the map of 3D environment by itself while calculating the best coverage path base on the already scanned map. After the robot finished building the map, it started to cover the map right away until coverage complete.

The experimental results of our system, integrating autonomous mapping, localization, navigation and coverage

path planning (CPP), demonstrate its effectiveness in achieving almost complete area coverage in a simulated indoor environment. The key outcomes of the experiments are summarized below:

A. Coverage Completeness

By performing efficient and precise mapping (with a resolution of 0.05 meters per pixel, length of grid cell of 0.2m, obstacle tolerance of 0.2m, goal tolerance of 0.2m), the turtlebot burger achieved about 85.36% coverage in the experiment, thanks to path planning and obstacle avoidance functions. But we can also decrease the obstacle tolerance, goal tolerance and length of grid cell to achieve higher coverage percentage but runtime will be longer.

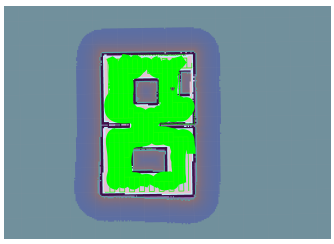

Fig. 5. Coverage Completeness

This result indicates:

Efficient Path Planning: By using appropriate path planning algorithms (A* algorithm), the robot effectively covered most of the cleaning area.

The A* algorithm is highly valued for its ability to guarantee optimal paths while effectively reducing the search space and improving efficiency through heuristic search. It excels in handling complex environments and dynamic obstacles, offering significant flexibility. Additionally, it can be combined with other heuristic methods or algorithms for better optimization.

B. Path Efficiency

The generated path using the heuristic in A* algorithm ensures the robot follows the most efficient route to cover the area while avoiding obstacles. Heuristic function (Manhattan distance) and distance Function (Euclidean distance) in A* algorithm can provide an admissible estimate of the cost to reach the goal and calculate the actual cost between nodes which accurately reflects the cost of movement in a grid. The algorithm also checks the four cardinal directions (up, down, left, right) to identify neighboring nodes. This ensures that the search space is limited to valid moves, reducing unnecessary expansions. Additionally, nodes with the lowest f-score will be expanded first, directing the search efficiently towards the goal.

The implementation of the A* algorithm for dynamic re-routing proved effective, allowing the robot to adapt to obstacles without significant deviations from the optimal path.

C. Obstacle Avoidance

The move-base navigation stack successfully detected and navigated around static obstacles placed within the environment. Real-time sensor data processing enabled the robot to avoid collisions and re-route dynamically, maintaining smooth and continuous coverage.

D. Running Time

The sum of the system running and covering time is about 15 minutes the overall running speed is respectively fast. The runtime can also be adjusted by changing the three parameters: length of grid cell of (default 0.2m), obstacle tolerance of (default 0.2m), goal tolerance of (default 0.2m). The larger the parameter the shorter the runtime.

E. Testing

Utilizing RViz and Gazebo for simulation testing provides a reliable and efficient way to validate the robot's performance in a controlled environment. This simulation approach allows for thorough testing of various scenarios and quick iteration of design improvements without the need for physical prototypes. It ensures that the robot's algorithms and behaviors are robust and well-tuned before deployment in real-world conditions.

IV. Summary

In this study, we designed and implemented a system integrating autonomous mapping, navigation, localization and coverage path planning (CPP) together with verifying its effectiveness in a simulated indoor environment. The system uses Gmapping SLAM for environment mapping and accurate localization and A* algorithm for dynamic path planning around obstacles. The experimental results show that the system is able to achieve at least 85.36% area coverage, efficient path planning, accurate positioning, and strong obstacle detection and avoidance capabilities. However, despite the satisfactory results, the system still has room for improvement in dynamic obstacle avoidance and algorithm optimisation.

Future Work

In order to further improve the system performance, especially in dynamic obstacle avoidance and algorithm optimisation, we have developed several plans for future work and optimisation. Firstly, for dynamic obstacle avoidance, we plan to integrate camera and depth sensors to enhance the robot's ability to sense the environment, capture and recognise the movement of dynamic obstacles. In addition, we would like to enable the robot to better adapt to dynamic environmental changes. For example, introducing machine learning methods, especially reinforcement learning and deep learning, which can train the robot to autonomously learn and optimise avoidance strategies in complex environments.

In terms of algorithm optimisation, we will evaluate and adopt more advanced SLAM algorithms to improve the accuracy and speed of mapping. In terms of path planning, we plan to combine the advantages of the A* algorithm

and the RRT algorithm to achieve global and local path optimisation by trying other algorithms. In addition, the coverage efficiency of large-scale areas can be significantly improved through area segmentation and parallel coverage by multiple robots.

Finally, in terms of data analysis and feedback mechanism, we will use big data analytics to process and analyse a large amount of experimental data to identify potential problems of the system and provide optimisation directions. By establishing a real-time feedback mechanism, using sensor data and system status monitoring, we are able to adjust algorithms and strategies in a timely manner to optimise system performance. By implementing these optimisation and expansion plans, the system will perform better in dynamic obstacle avoidance, algorithm optimisation and real-world applications. This will provide stronger technical support for tasks such as robotic autonomous cleaning, inspection and maintenance, as well as promote the development and application of intelligent robotics.

REFERENCES

[1] Bormann R, Jordan F, Hampp J, et al. Indoor coverage path planning: Survey, implementation, analysis[C]//2018 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2018: 1718-1725.